

PolyFlop8 ALU

Unit Test Card

Comprehensive Verification of Arithmetic Logic Unit

Document ID:	TC-ALU-001
Revision:	1.0
Date:	January 5, 2026
Test Level:	Unit/Block Level
Test Engineer:	Artem Horiunov
Status:	DEVELOPMENT

1 TEST OVERVIEW

1.1 Unit Under Test (UUT)

Component	Arithmetic Logic Unit (ALU)
VHDL Entity	ALU
Source File	Processor.txt (first entity)
Test Environment	ModelSim/QuestaSim 2023.4, VHDL Testbench
Clock Frequency	100MHz (10ns period)
Time Resolution	1ns

1.2 Scope

This test card defines the verification procedure for the PolyFlop8 ALU component. The ALU performs all arithmetic and logical operations, generates status flags, and selects operands via internal multiplexers.

1.3 Test Objectives

- Verify all arithmetic operations produce correct 8-bit results
- Validate flag generation logic (H, V, N, C, Z)
- Confirm proper multiplexer selection for operand sources
- Test 2's complement handling and overflow detection
- Ensure 100% code coverage of ALU behavioral description

2 REQUIREMENTS COVERAGE

Req ID	Requirement Description	Test Case ID	Status
TR-ALU-01	Verify all arithmetic/logical operations produce correct 8-bit results	TC-ALU-001-01	
TR-ALU-02	Verify flags (H, V, N, C, Z) are correctly asserted for each operation	TC-ALU-001-02	
TR-ALU-03	Verify alu_mux_sel_a correctly selects between data sources	TC-ALU-001-03	
TR-ALU-04	Verify alu_mux_sel_b correctly selects between data sources	TC-ALU-001-04	
TR-ALU-05	Verify 2's complement handling in NEG and SUB operations	TC-ALU-001-05	

Table 1: Requirements Coverage Matrix

3 DETAILED TEST CASES

3.1 Test Case TC-ALU-001-01: Basic Arithmetic Operations

Requirement: TR-ALU-01

Purpose: Verify all arithmetic/logical operations produce correct 8-bit results.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
1.1	ADD, no carry	alu_sel = "0000" op_a = 0x25 op_b = 0x37 cin = '0'	result = 0x5C flags = 0x00 (No flags set)
1.2	ADC (with carry)	alu_sel = "0001" op_a = 0x25 op_b = 0x37 cin = '1'	result = 0x5D flags = 0x00 (No flags set)
1.3	SUB, no borrow	alu_sel = "0010" op_a = 0x50 op_b = 0x30 cin = '0'	result = 0x20 flags = 0x00 (No flags set)
1.4	SBC (with borrow)	alu_sel = "0011" op_a = 0x50 op_b = 0x30 cin = '1'	result = 0x1F flags = 0x00 (No flags set)
1.5	AND logical op	alu_sel = "0100" op_a = 0x3C op_b = 0x0F	result = 0x0C flags = 0x00 (No flags set)
1.6	OR logical op	alu_sel = "0101" op_a = 0x3C op_b = 0x0F	result = 0x3F flags = 0x00 (No flags set)
1.7	XOR logical op	alu_sel = "0110" op_a = 0x3C op_b = 0x0F	result = 0x33 flags = 0x00 (No flags set)
1.8	MOV (pass B)	alu_sel = "0111" op_a = 0xAA op_b = 0x55	result = 0x55 flags = 0x00 (No flags set)
1.9	LDI (pass A)	alu_sel = "1000" op_a = 0xAA op_b = 0x55	result = 0xAA flags = 0x00 (No flags set)
1.10	COM (1's complement)	alu_sel = "1001" op_a = 0x55 op_b = 0x00	result = 0xAA flags = 0x02 (C=1, Z=0)
1.11	NEG (2's complement)	alu_sel = "1010" op_a = 0x01 op_b = 0x00	result = 0xFF flags = 0x02 (C=1, Z=0)

3.2 Test Case TC-ALU-001-02: Flag Generation

Requirement: TR-ALU-02

Purpose: Verify flags (H, V, N, C, Z) are correctly asserted.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
2.1	Zero Flag (Z) on ADD	alu_sel = "0000" op_a = 0x00 op_b = 0x00	result = 0x00 flags = 0x01 (Z=1)
2.2	Carry Flag (C) on ADD overflow	alu_sel = "0000" op_a = 0xFF op_b = 0x01	result = 0x00 flags = 0x03 (C=1, Z=1)
2.3	Half Carry Flag (H) on ADD	alu_sel = "0000" op_a = 0x0F op_b = 0x01	result = 0x10 flags = 0x10 (H=1)

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
2.4	Overflow Flag (V) on signed ADD	alu_sel = "0000" op_a = 0x7F op_b = 0x01	result = 0x80 flags = 0x0C (V=1, N=1)
2.5	Negative Flag (N) on result MSB=1	alu_sel = "0010" op_a = 0x00 op_b = 0x01	result = 0xFF flags = 0x06 (N=1, C=1)
2.6	Flags on logical AND	alu_sel = "0100" op_a = 0x80 op_b = 0x80	result = 0x80 flags = 0x04 (N=1)
2.7	All flags on complex op	alu_sel = "0010" op_a = 0x80 op_b = 0x81 cin = '1'	result = 0xFE flags = 0x0E (V=1, N=1, C=1)

3.3 Test Case TC-ALU-001-03: MUX A Selection

Requirement: TR-ALU-03

Purpose: Verify alu_mux_sel_a selects correctly.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
3.1	MUX A selects RegFile A (sel=0)	alu_mux_sel_a = '0' regfile_data_a = 0xAA pc_data = 0x55 alu_sel = "1000"	result = 0xAA (Passes regfile data)
3.2	MUX A selects PC (sel=1)	alu_mux_sel_a = '1' regfile_data_a = 0xAA pc_data = 0x55 alu_sel = "1000"	result = 0x55 (Passes PC data)
3.3	MUX A with arithmetic op	alu_mux_sel_a = '0' regfile_data_a = 0x10 pc_data = 0x20 op_b = 0x05 alu_sel = "0000"	result = 0x15 (10h + 05h = 15h)

3.4 Test Case TC-ALU-001-04: MUX B Selection

Requirement: TR-ALU-04

Purpose: Verify alu_mux_sel_b selects correctly.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
4.1	MUX B selects RegFile B (sel=0)	alu_mux_sel_b = '0' regfile_data_b = 0xAA rs_imm_in = 0x55 alu_sel = "0111"	result = 0xAA (Passes regfile data)
4.2	MUX B selects Immediate (sel=1)	alu_mux_sel_b = '1' regfile_data_b = 0xAA rs_imm_in = 0x55 alu_sel = "0111"	result = 0x55 (Passes immediate data)
4.3	MUX B with ADD op	alu_mux_sel_b = '1' op_a = 0x10 regfile_data_b = 0xAA rs_imm_in = 0x05 alu_sel = "0000"	result = 0x15 (10h + 05h = 15h)

3.5 Test Case TC-ALU-001-05: 2's Complement Operations

Requirement: TR-ALU-05

Purpose: Verify 2's complement handling.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
5.1	NEG of 0x00 (Edge case)	alu_sel = "1010" op_a = 0x00 op_b = 0x00	result = 0x00 flags = 0x00 (No flags)
5.2	NEG of 0x80 (Min signed value)	alu_sel = "1010" op_a = 0x80 op_b = 0x00	result = 0x80 flags = 0x0A (V=1, C=1)
5.3	NEG of positive number	alu_sel = "1010" op_a = 0x01 op_b = 0x00	result = 0xFF flags = 0x02 (C=1)
5.4	SUB overflow (positive)	alu_sel = "0010" op_a = 0x7F op_b = 0x80	result = 0xFF flags = 0x0E (V=1, N=1, C=1)
5.5	SUB overflow (negative)	alu_sel = "0010" op_a = 0x80 op_b = 0x7F	result = 0x01 flags = 0x08 (V=1)
5.6	SBC with borrow and overflow	alu_sel = "0011" op_a = 0x80 op_b = 0x7F cin = '1'	result = 0x00 flags = 0x09 (V=1, Z=1)
5.7	SUB of equal values	alu_sel = "0010" op_a = 0x55 op_b = 0x55	result = 0x00 flags = 0x01 (Z=1)

4 TEST BENCH ARCHITECTURE

4.1 VHDL Testbench Structure

```
-- ALU Testbench (alu_tb.vhd)
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity alu_tb is
end alu_tb;

architecture Behavioral of alu_tb is
    -- Component Declaration
    component ALU is
        Port (
            pc_data          : in  STD_LOGIC_VECTOR (7 downto 0);
            regfile_data_a   : in  STD_LOGIC_VECTOR (7 downto 0);
            rs_imm_in        : in  STD_LOGIC_VECTOR (7 downto 0);
            regfile_data_b   : in  STD_LOGIC_VECTOR (7 downto 0);
            alu_mux_sel_a    : in  STD_LOGIC;
            alu_mux_sel_b    : in  STD_LOGIC;
            alu_sel          : in  STD_LOGIC_VECTOR (3 downto 0);
            cin              : in  STD_LOGIC;
            result           : out STD_LOGIC_VECTOR (7 downto 0);
            flags            : out STD_LOGIC_VECTOR (4 downto 0)
        );
    end component;
end component;
```

```

-- Test Signals
signal pc_data_tb      : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
signal regfile_data_a_tb : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
signal rs_imm_in_tb    : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
signal regfile_data_b_tb : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
signal alu_mux_sel_a_tb : STD_LOGIC := '0';
signal alu_mux_sel_b_tb : STD_LOGIC := '0';
signal alu_sel_tb      : STD_LOGIC_VECTOR (3 downto 0) := (others => '0');
signal cin_tb          : STD_LOGIC := '0';
signal result_tb       : STD_LOGIC_VECTOR (7 downto 0);
signal flags_tb        : STD_LOGIC_VECTOR (4 downto 0);

-- Clock for timing reference (ALU is combinational, but useful for sequencing)
signal clk_tb : STD_LOGIC := '0';
constant CLK_PERIOD : time := 10 ns;

begin
  -- Unit Under Test Instantiation
  UUT: ALU port map (
    pc_data      => pc_data_tb,
    regfile_data_a => regfile_data_a_tb,
    rs_imm_in    => rs_imm_in_tb,
    regfile_data_b => regfile_data_b_tb,
    alu_mux_sel_a => alu_mux_sel_a_tb,
    alu_mux_sel_b => alu_mux_sel_b_tb,
    alu_sel      => alu_sel_tb,
    cin          => cin_tb,
    result       => result_tb,
    flags        => flags_tb
  );

  -- Clock Generation Process
  clk_process: process
  begin
    clk_tb <= '0';
    wait for CLK_PERIOD/2;
    clk_tb <= '1';
    wait for CLK_PERIOD/2;
  end process;

  -- Main Test Stimulus Process
  stim_proc: process
  begin
    report "Starting ALU Test Bench";
    wait for 100 ns;

    -- Test Case 1.1: ADD operation
    report "Test Case 1.1: ADD 0x25 + 0x37";
    alu_sel_tb <= "0000"; -- ALU_ADD
    regfile_data_a_tb <= x"25";
    regfile_data_b_tb <= x"37";
    cin_tb <= '0';
    wait for CLK_PERIOD;
    assert result_tb = x"5C"
      report "ADD result incorrect: Expected 0x5C, got " &
        to_hstring(result_tb) severity error;
    assert flags_tb = "00000"
      report "ADD flags incorrect: Expected 0x00, got " &
        to_hstring(flags_tb) severity error;

    -- Continue with all other test cases...

    -- Final Report

```

```
        report "All ALU test cases completed successfully!";  
        wait;  
    end process;  
  
end Behavioral;
```

4.2 Test Sequence

1. **Initialization:** Reset all signals to known state
2. **Basic Operations:** Test ADD, SUB, AND, OR, XOR, MOV, LDI
3. **Flag Testing:** Systematically test each flag condition
4. **MUX Testing:** Verify both multiplexers with all select options
5. **Edge Cases:** Test boundary values and overflow conditions
6. **Randomized Testing:** Additional random inputs for robustness

4.3 Monitoring and Logging

- All test results logged to simulation transcript
- Waveform database captured for debugging
- Coverage metrics collected (statement, branch, toggle)
- Assertion failures trigger immediate error reporting
- Test summary report generated at simulation end

5 PASS/FAIL CRITERIA

5.1 Success Criteria

Criteria	Description
Functional	All test cases execute without assertion failures
Timing	No timing violations (ALU is combinational)
Coverage	100% statement coverage, 100% branch coverage achieved
Consistency	Identical results across multiple simulation runs
Documentation	All test results properly documented and signed off

5.2 Failure Conditions

- Any single assertion failure during simulation
- Code coverage less than 100% for ALU behavioral code
- Timing violations or simulation warnings
- Inconsistent results between simulation runs
- Missing or incomplete test documentation

5.3 Coverage Goals

- **Statement Coverage:** 100% of ALU process statements executed
- **Branch Coverage:** 100% of case statement branches taken
- **Condition Coverage:** All flag calculation conditions tested
- **Toggle Coverage:** All signal bits toggled 0→1 and 1→0
- **FSM Coverage:** N/A (ALU is combinational)

6 SPECIAL TEST CONSIDERATIONS

6.1 Edge Cases

- Test all arithmetic operations with 0x00 and 0xFF operands
- Verify proper handling of signed vs unsigned overflow
- Test NEG operation with minimum signed value (0x80)
- Verify carry chain behavior through all bit positions
- Test all possible multiplexer combinations

6.2 Timing Considerations

- ALU is purely combinational - no clock dependencies
- Monitor for glitches or hazards during input transitions
- Verify stable outputs within specified propagation delay
- Test with minimum, typical, and maximum delay models

6.3 Integration Considerations

- Verify interface compatibility with Register File
- Test handshake signals with Control Unit
- Validate flag output format matches Status Register expectations
- Confirm result bus loading and fanout capabilities